

"""

rules.py – computes parking restriction windows for a Spurs stadium event day.

Based on published Haringey Council / Tottenham Hotspur guidance:

- Wider roads (e.g. Worcester Avenue) can close from 8am on matchdays.
- Park Lane and side roads closest to the ground close from ~3 hours before kick-off.
- The general road-closure zone (Phase 1) typically tightens from ~2 hours before the event.
- Phase 2 (tightest closure, no vehicles except emergency services) runs from ~1 hour before the event to ~15 minutes after.
- Restrictions/closures generally clear by ~1 hour after full time, at the Police Match Commander's discretion – always treat this as a minimum, not a guarantee.
- A Penalty Charge Notice (PCN) for parking in a suspended bay or restricted zone is currently £160 (reduced to £80 if paid within 14 days) – figures can change, always confirm with Haringey/Enfield.

IMPORTANT: These are default assumptions for planning purposes only. Exact closure times vary by event and are only confirmed ~7 days ahead via Haringey's own advertising of bay suspensions. Always treat the computed "move by" time as the LATEST safe time, and encourage users to check official confirmation nearer the date.

"""

```
from datetime import datetime, timedelta
```

```
# Conservative defaults (hours), editable per-deployment
```

```
DEFAULT_RULES = {
```

```
    "closure_starts_hours_before": 3, # be safe: earliest roads (Park Lane) start closing
```

```
    "restriction_ends_hours_after": 1, # typical clearance after final whistle
```

```
    "pcn_amount_full": 160,
```

```
    "pcn_amount_reduced": 80,
```

```
    "pcn_reduced_window_days": 14,
```

```
}
```

```
def compute_restriction_window(event_date: str, kickoff_time: str, rules: dict = None) -> dict:
```

```
    """
```

```
    event_date: 'YYYY-MM-DD'
```

```
    kickoff_time: 'HH:MM' 24hr
```

```
    Returns dict with move_by (datetime),
```

```

restriction_start, restriction_end.
    """
    rules = rules or DEFAULT_RULES
    kickoff_dt = datetime.strptime(f"
{event_date} {kickoff_time}", "%Y-%m-%d
%H:%M")

    restriction_start = kickoff_dt -
timedelta(hours=rules["closure_starts_hours
_before"])
    restriction_end = kickoff_dt +
timedelta(hours=rules["restriction_ends_hou
rs_after"])

    return {
        "kickoff": kickoff_dt,
        "move_by": restriction_start, #
latest safe time to have moved the car
        "restriction_start":
restriction_start,
        "restriction_end": restriction_end,
    }

def notification_times(event_date: str,
kickoff_time: str, lead_time_hours: int,

reminder_hours_before: int, rules: dict =
None) -> dict:
    """
    Returns when to send the 'advance'
notice and the 'reminder' notice,
    both anchored to the move-by time (not
kickoff), so a subscriber's

```

```
    lead_time_hours means "hours before I
    need to have moved my car."
    """
    window =
    compute_restriction_window(event_date,
    kickoff_time, rules)
    move_by = window["move_by"]

    return {
        **window,
        "advance_notice_at": move_by -
        timedelta(hours=lead_time_hours),
        "reminder_at": move_by -
        timedelta(hours=reminder_hours_before),
    }
```